Altiora · 未来的我

数字孪生分身 + 代价测算引擎

技术文档

全国高校 Vibe Coding 开发挑战赛 · B 组

赛道 8：个人效能与赛博具身智能仿真

提交日期：2026 年 11 月

目录

章节	内容	页码
1	作品概述	3
2	架构设计	4
3	意图控制策略（重点）	7
4	核心功能与确定性逻辑	14
5	工程质量与安全	17
6	可复现性论证	19
7	测试方案	22
8	部署方案	23

1. 作品概述

1.1 一句话定位

市面上的 AI 助手、日历、习惯养成 App 都在替用户做决定。Altiora 不替你决定——它只把每个选择的代价算清楚，让你自己挑。

1.2 解决的痛点

- 现有助手替用户做选择，用户失去掌控感，完成率一掉就想「重开」。
- 日程冲突没有「代价」视角，只有「哪件更重要」的粗暴排序。
- AI 生成内容不可信：模型会编时间、编数字、给鸡汤，用户无法信任。

1.3 核心创新

代价账单代替记分板：冲突时给出 A/B/C 三个选项，每个选项的得与失都摆出来，不替用户选。
LLM 只做翻译，不做计算：所有时间差、金钱、冲突分钟数由本地确定性代码算出，LLM 只把自然语言翻成结构化字段、把结果说成人话。
档案不是摆设：精力高峰、睡眠硬约束、软肋、目标都真实影响排程——改一个参数，今日方案肉眼可见地变。

2. 架构设计

2.1 三段式架构

整体分为三段，第二段（计算器）不含任何概率成分，同一输入永远同一输出。这是用户能信任它的唯一理由。

阶段	职责	是否调用 LLM
① AI 翻译官	自然语言 → 结构化字段（意图/事件/记忆/目标/情绪）	是，失败自动退回本地规则
② 计算器	冲突检测、代价账单、今日方案排程	否，纯确定性代码
③ AI 表达	把情绪与计算结果说成人话	是，失败回退确定性文案

表 2-1 三段式架构

2.2 前后端职责划分

前端（单 HTML 文件）：所有状态、记忆、排程、冲突计算都在浏览器内完成。双击 HTML 即可用文字与 AI，不需要 Python。
后端（FastAPI）：仅提供两个能力——语音识别（Vosk 离线模型）和可选的 LLM 代理。不存用户数据，不写 Key 日志。

数据流

用户输入 → LLM 翻译（directUnderstand）→ 结构化字段 → 本地计算器（detectConflicts / buildTodayPlan）→ 结果 → LLM 表达（directSay）→ 展示。LLM 永远不接触数字计算。

2.3 关键设计原则

- LLM 边界写死在代码里：intent 不是 emotion/chat 时，模型生成的 reply 被直接丢弃。
- 没说结束时间就追问：event 为 null 时不替用户编时间，宁可问一句。
- 双模式自动切换：无 Key 走本地规则，有 Key 走 AI 在线，功能都完整。
- BYOK（自带 Key）：Key 只存浏览器 localStorage，请求由浏览器直连厂商，服务端不落盘。

3. 意图控制策略

这是本作品的核心。我们用三层锁把 LLM 锁在「翻译官」的角色里，防止它越界做计算、做决策、说鸡汤。

3.1 三层锁总览

层级	机制	作用	实现位置
第一层	SYSTEM Prompt 边界	从输入侧告诉模型能做什么、不能做什么	llm.py SYSTEM / 前端 LLM_SYS
第二层	_BANNED 正则	过滤客服腔和鸡汤句	llm.py _BANNED / 前端 LLM_BANNED
第三层	reply 丢弃 + 数字校验	schedule/safety 场景强制丢弃 reply；say 阶段数字必须来自	llm.py 双阶段 / 前端 directUnderstan

表 3-1 意图控制三层锁

3.2 第一层：SYSTEM Prompt 边界

SYSTEM prompt 明确列出 7 种意图（safety/emotion/schedule/memory/goal/social/chat），并对每种 intent 的 event/memory/goal/social 字段做了抽取规则。最关键的是「绝对不能做」清单：

```
【绝对不能做】
- 不计算时间重叠、时长、金钱、代价
- 不替用户选方案，不说「你应该选 A」
- 不编造用户没说的地点、时间、人物
- 不说「加油」「你可以的」「相信自己」，不喊口号
- 不复述用户原话，不用客服腔
```

reply 的写法也被严格约束：intent=schedule 且 event 完整时 reply 必须为 null（账还没算，不能抢答）；intent=safety 时 reply 为 null（前端有固定安全话术）。

3.3 第二层：_BANNED 正则过滤

即使 SYSTEM prompt 写了禁止，模型仍可能输出客服腔。所以在代码层加了一道正则，命中即作废。

```
_BANNED = re.compile(
    r"嗯[，,]\s*我在听|我在听|你接着说|我在这儿|我都在|"
    r"会好起来的|会好一点|加油|你可以的|相信自己|"
    r"慢慢来|不要放弃|你能做到"
)
# 命中 → reply = None，前端用本地兜底话术
```

前后端各有一份完全相同的正则，因为 LLM 调用有两条路径：浏览器直连（directUnderstand）和服务端代理（understand）。两条路径都要过滤。

3.4 第三层：reply 丢弃 + 数字校验

3.4.1 schedule/safety 场景强制丢弃 reply

完整日程的账必须由计算器算完再说，LLM 不能抢答。安全场景由前端固定话术处理，不让模型自由发挥。

```
# 服务端 llm.py
if intent in ("safety",) or (intent == "schedule" and event):
    reply = None

# 前端 directUnderstand（同样逻辑，双保险）
if intent == "safety" or (intent == "schedule" and event):
    reply = None
```

3.4.2 say 阶段数字校验

第二段翻译官（directSay / say）把计算器的结果说成人话。为防止模型编造数字，我们校验：输出中出现的所有数字，必须是 facts 里已有的数字，多一个就整句作废。

```
# 前端 directSay
const allowed = new Set((JSON.stringify(facts).match(/\d+(?:\.\d+)?/g) || []));
const nums = text.match(/\d+(?:\.\d+)?/g) || [];
if (nums.some(n => !allowed.has(n))) return ""; // 编造数字 → 作废
```

服务端 say() 同样实现了数字集合校验（_numbers 函数），双路径一致。

3.5 失败回退机制

任何一层校验失败，都不会让用户看到空白或错误——而是回退到本地确定性文案。

- directUnderstand 返回 null → 前端用 parseEventsFromText 本地解析，或追问时间。
- directSay 返回空 → 前端用拼好的确定性文案（如「『考研复习 14:00-16:00』加进时间轴了，没冲突」）。
- explainPlan 失败 → 用 planCaption() 拼一句说清「为什么这么排」。

核心原则：LLM 是锦上添花，不是功能前提。没有 Key 或 Key 失效，所有功能依然完整可用。

4. 核心功能与确定性逻辑

4.1 冲突检测与代价账单

detectConflicts 遍历同一天的事件两两比较，重叠分钟数 > 0 即为冲突。makeConflict 为每对冲突生成 A/B/C 三个选项：

选项	含义	代价示例
A	去优先级高的那件	损失家教 2 小时 × 80 元 = 160 元
B	去优先级低的那件	放弃重要的事，事后可能后悔

选项	含义	代价示例
C	把低优先级的顺延	连轴转，可能破睡眠硬约束

表 4-1 冲突代价账单三选项

注意：makeConflict 里的金钱计算（wage × dur）、优先级判断、顺延时间（hi.end + 30 分钟）全部是本地计算，LLM 完全不参与。

4.2 今日方案排程（buildTodayPlan）

这是「档案不是摆设」的核心证明。规则：

- 课程/会议/睡眠 = 固定项，原地不动；学习/兼职 = 可动项。
- 可动项按「优先级小者先 → 时长长的先」排序。
- 在精力高峰时间窗内贪心塞入，撞到固定项就绕过，塞不下保持原位。

改档案里的 preferred_focus_hours，方案肉眼可见地变：

```
档案精力高峰 = "14:00–17:00"
  → 「考研复习」从 15:00-17:00 挪到 14:00-16:00（moved=true）

档案精力高峰 = "08:00–11:00"（此时 08:30-10:10 有课占窗）
  → 「考研复习」保持原位（moved=false）

档案精力高峰 = 未填
  → 方案按原时间排，不优化
```

4.3 记忆三层

层	存什么	保留多久	谁用
长期记忆 (profile)	目标、软肋、睡眠硬约束、规矩	一直	计算器输入，只有明说才写
聊天记忆	最近 3 轮对话原文	超 6 条滚掉	只接半句（「就那个」「三点」）
日记	每行「日期：摘要」	最多 180 行	隔 3 天以上回来才翻

表 4-2 记忆三层设计

5. 工程质量与安全

5.1 DNS 重绑定防护

用户自填的 LLM Base URL 可能被用来探测内网。我们做了两层防护：

- 第一层：字符串前缀过滤，挡掉 localhost、127.x、10.x、192.168.x 等。
- 第二层：DNS 解析后逐一检查返回 IP，若落在私网/回环/保留范围则拒绝（防 DNS 重绑定）。

```
def _safe_base(base):
    u = urlparse(base)
    if u.scheme != "https" or not u.hostname: return False
    # 第一层：字符串前缀
    if any(h == p or h.startswith(p) for p in _PRIVATE_HOSTS): return False
    # 第二层：DNS 解析后查 IP
```

```
infos = socket.getaddrinfo(h, u.port or 443, type=socket.SOCK_STREAM)
for info in infos:
    if _is_private_ip(info[4][0]): return False
return True
```

5.2 BYOK 安全设计

- Key 只存浏览器 localStorage，服务端不落盘、不写日志、不进仓库。
- 请求由浏览器直连厂商 API，不经过服务端中转。
- 服务端 .env 模式仅用于单人自用；开放给他人时强制 BYOK。

5.3 输入校验

- event 必须有 name + 合法 start/end（HH:MM），否则丢弃。
- 时间区间必须 end > start，否则 event = null。
- 所有字段有长度上限（name ≤20，memory ≤60，goal ≤60）。
- extra_headers 做头注入防护：过滤 Host/Content-Length 等危险头，限制长度。

6. 可复现性论证

评审标准中「过程可复现性与演进链路」占 25%。我们从四个维度保证可复现。

6.1 细粒度 Git 提交历史

禁止「initial commit 包含全部功能」。每次提交对应一次与 AI 的核心对话或一个功能模块的落地。近期提交示例：

提交哈希	说明
0480027	feat(frontend): 时间轴每条日程加删除按钮
a332469	fix(frontend): LLM 没抽出 event 时用本地时间解析兜底
6c504f0	fix(frontend): tryFillPending 别在用户说了日期时拿旧日期乱填
deab5cb	feat(frontend): 日记以小卡片展示
0b77f5f	fix(frontend): 保存的 Key 显示圆点占位 + 语音不可用给出具体方案

表 6-1 细粒度提交示例（完整历史见 git log）

6.2 Prompt 历史快照

prompt-snapshots/ 目录保存了 6 个核心功能迭代的完整对话链截图，每个都包含「用户提问 → AI 输出 → 决策/实测」：

- 迭代 1：目标规划 + 自适应重规划闭环
- 迭代 2：功能分层 + 记忆库 vs 轻量方案选型
- 迭代 3：产品表现形式（小白用户提问 → 通俗类比）
- 迭代 4：LLM 配置面板 + 本地 TTS（含安全决策）

- 迭代 5：修 Bug——长期目标存成整句（七步抽取规则 + 实测对照表）
- 迭代 6：记忆分三层（长期/聊天/日记）

远超比赛要求的「≥3 个核心功能迭代完整 Prompt 链」。PROMPT_LOG.md 记录了 Prompt 迭代与「LLM 忽略档案」bug 的调试全过程。

6.3 自检脚本

自检.py 提供十项检查：Python 版本、四个依赖、语音模型完整性、前端关键控件、页面无真实 API Key、服务启动、识别接口连通。不联网、不调大模型、不消耗额度。全过退出码 0。

6.4 确定性断言

排程核心（focusWindowOf / buildTodayPlan / planCaption）可脱离浏览器环境单独断言。实测：

```
精力高峰 = "14:00–17:00" → 考研复习挪到 14:00-16:00  √
精力高峰 = "08:00–11:00"（被课占）→ 不动              √
精力高峰 = 未填 → 不优化                              √
ALL PASS √
```

同一输入永远同一输出，证明排程逻辑完全确定、可复现。

7. 测试方案

#	操作	预期结果
1	说「今天没睡好」	AI 用一句话接住情绪，不出表格
2	「我明天下午要和导师讨论」	听懂意图但时间不全 → 追问几点到几点，不编时间
3	「周五下午3点到5点去家教」	抽成事件排进时间轴，算出冲突与代价
4	「记住，我晚上容易刷手机」	写入本地画像，冲突时代价栏会出现
5	改精力高峰为 14:00–17:00，生成今日方案	可动脑力活被挪进高峰窗
6	配自己的 LLM Key	AI 在线模式，理解更准，回复更像人话
7	删除日程	时间轴 hover 出 ×，点击删除，冲突重算

表 7-1 核心链路测试用例

8. 部署方案

8.1 零依赖模式（推荐评委使用）

双击 Altiora-未来的我.html 即可。文字对话、排程、冲突代价、记忆、今日方案全部在前端完成，不需要 Python、不需要联网。

8.2 语音模式

```
pip install -r requirements.txt
```

```
python3 scripts/download_vosk.py    # 仅首次，下载语音模型
python server.py                    # 启动后打开 http://localhost:8000
```

8.3 AI 在线模式

页面右上角点状态标签 → 填自己的 API Key → 保存。Key 只存本机浏览器，请求直连厂商。支持 DeepSeek / 通义 / 火山方舟等 OpenAI 兼容接口。